

Chapter 8

Hashing

哈希

Ajou University

目录

符号表抽象数据类型

静态哈希 [static hashing]

- 哈希表

- 哈希函数

- 溢出处理

动态哈希 [Dynamic hashing]

- 可扩展哈希算法

- 线性哈希

符号表

- 符号表 [Symbol table]
 - 一组名称属性对
 - 示例
 - 同义词词典
 - 编译符号表
 - 词库
 - 操作
 - 确定表中是否有特定名称
 - 读取该名称的属性
 - 修改名称属性
 - 插入新名称及其属性
- 哈希
 - 有效插入、搜索和删除符号的技术

静态哈希

❖ 静态哈希 [Static hashing]

- 标识符存储在一个固定大小的表中，称为哈希表[hashing table]

❖ 哈希表 ht

- b buckets 和 s slots
- $ht[0], \dots, ht[b-1]$

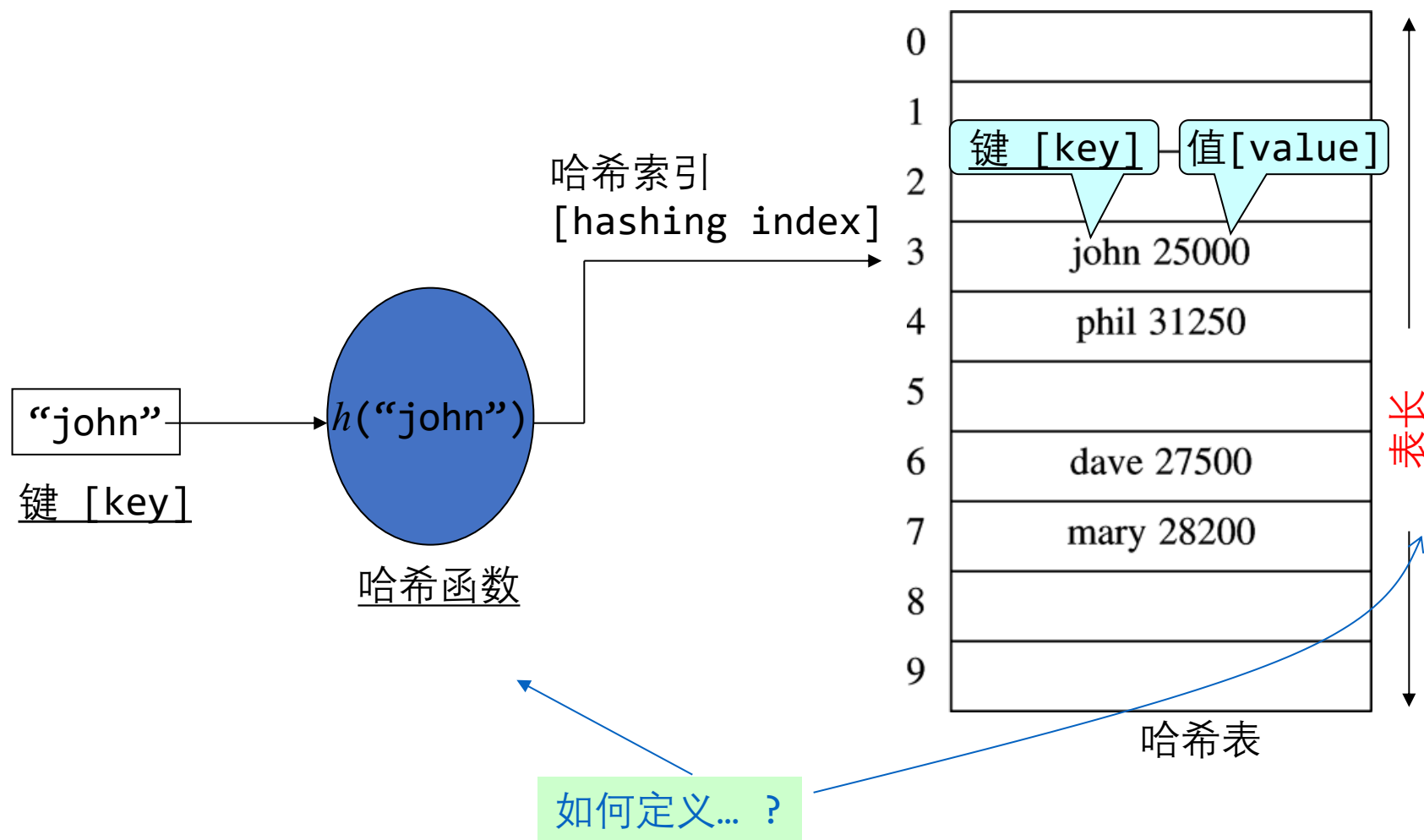
[0]	<i>bucket</i>	
[1]	<i>slot</i>	<i>slot</i>
[2]		
[3]		
[4]		
[5]		
[6]		

❖ 哈希函数 $h(x)$

- 标识符[identifier] $x \rightarrow$ 哈希表中的地址

最坏情况下的 **Search**, **Insert**, 和 **Delete** 是 $O(b \times s)$.
预计时间为 $O(1)$.

哈希表



术语

- 标识符密度 [Identifier density] 是比值 n/T , 其中 n 是表中标识符的数量, T 是可能的标识符总数。
- 负载因子 [Loading factor] $\alpha = \frac{n}{s \times b}$
- 如果 $h(i_1) = h(i_2)$, 则标识符 i_1 和 i_2 关于哈希函数 h 是同义词 [synonyms]
- 当将新标识符 i 哈希到已满的桶时, 发生溢出 [overflow]
- 当两个不同的标识符被哈希到同一桶时, 发生冲突 [collision]

哈希示例

❖ 哈希表 [Hash table]

$$b = 26, s = 2, n = 10$$

$$\text{负载因子 [Loading factor]} = 10 / 52 = 0.19$$

❖ Id

“acos”, “define”, “float”, “exp”, “char”, “atan”, “ceil”,
“floor”, “clock”, “ctime”

哈希函数 $h(x)$: x 的第一个字符 - ‘a’

$h(\text{acos})=0$	$h(\text{define})=3$	$h(\text{float})=5$	$h(\text{exp})=4$
$h(\text{char})=2$	$h(\text{atan})=0$	$h(\text{ceil})=2$	$h(\text{floor})=5$
$h(\text{clock})=2$	$h(\text{ctime})=2$		

acos 的同义词?

clock的同义词?

溢出?

哈希函数

❖ 哈希函数 h 的要求

- 高效计算
- 最小化冲突次数
- 分布无偏

❖ 统一哈希函数

- 对于所有 i , 哈希函数 $h(x)$ 等于 i 的概率为 $1/b$

❖ 示例

- 平方取中法
- 除余法
- 折叠法
- 数字分析法

平方取中法

哈希函数, h , 通过取标识符的平方计算

通过从平方值的中间部分选取适当数量的位来获取 bucket 地址

→ 若哈希表大小为 2^r , 则选取 r 位

- 平方值的中间位通常依赖于键值的所有字符
- 即使部分字符相同, 不同键值仍大概率生成不同的哈希地址

表尺寸=16

$r=4$

Key=4562

$(4562 * 4562) \% 65536 = 36932 = 1001000001000100$

哈希位置 = 1

Key=3981

$(3981 * 3981) \% 65536 = 54185 = 1101001110101001$

哈希位置 = 14

除余法 (1)

❖ 使用调制 (%) 运算符

- $h_D(x) = x \% M$
- Bucket : $0 \sim (M-1)$

❖ M 的选择

- $M = 2^p$, $h_D(x)$ 仅取 x 的低 p 位
- 若 $M = 2^p - 1$ 且 x 按 2^p 进制解析, 则重排 x 的字符不会改变哈希值
- 选择 M 时, 一个不太接近 2 的精确幂的素数通常是更好的选择

$$\begin{aligned} M=8=2^3 \quad x=13=01101_{(2)} \quad h(x)=5 \\ x=17=10001_{(2)} \quad h(x)=1 \end{aligned}$$

$$\begin{aligned} M=7=2^3-1 \quad \text{String "abcd"} \\ x = 97 \cdot 8^3 + 98 \cdot 8^2 + 99 \cdot 8 + 100 = 56828 \quad h(x)=2 \\ \\ \text{String "badc"} \\ x = 98 \cdot 8^3 + 97 \cdot 8^2 + 100 \cdot 8 + 99 = 57283 \quad h(x)=2 \end{aligned}$$

除余法 (2)

- 若除数为偶数，奇数键值将哈希到奇数主桶，偶数键值将哈希到偶数
- 若除数为奇数，奇数（或偶数）键值可能哈希到任意主桶

$M=14$, $20\%14 = 6$, $30\%14 = 2$, $8\%14 = 8$
 $15\%14 = 1$, $3\%14 = 3$, $23\%14 = 9$

$M=15$, $20\%15 = 5$, $30\%15 = 0$, $8\%15 = 8$
 $15\%15 = 0$, $3\%15 = 3$, $23\%15 = 8$

折叠法

❖ 折叠

- 将键 X 分割为若干部分，除最后一部分外，其余部分长度相等，
- 然后通过适当方式相加，得到哈希地址

❖ 移位折叠：将所有字符符合二为一

e. g.) 5 元组 id $x = (x_1:123, x_2:203, x_3:241, x_4:112, x_5:20)$

$$x_1 + x_2 + x_3 + x_4 + x_5 = 699$$

01111011	11001101	11110001	01110000	00010100
----------	----------	----------	----------	----------

❖ 在边界处折叠：在添加之前先反转每一个分区

e. g.) 反转 x_2, x_4 并且添加他们

$$x_1 + \text{反转}(x_2) + x_3 + \text{反转}(x_4) + x_5$$

01111011	10110011	11110001	00001110	00010100
----------	----------	----------	----------	----------

$$(123) + (179) + (241) + (14) + (20) = 557$$

数字分析法

- ❖ 所有 IDs 是事先知道的作为一个预先作为静态文件的方法
 - 将标识符 (ID) 转换为以基数 r 表示的数字
 - 这些数位的分布不得有异常的高峰或低谷
 - 用剩余数位进行哈希计算
 - 这些数位的分布不得有异常的高峰或低谷

将键转换为整数

```
unsigned int stringToInt1(char *key)
{
    int number = 0;
    while (*key)
        number += *key++;
    return number;
}
```

E.g. “AZF”
A = 65
Z = 90
F = 70
→ `stringToInt1(“AZF”) = 225`

```
unsigned int stringToInt2(char *key)
{
    int number = 0;
    while (*key)
    {
        number += *key++;
        if(*key) number += ((int) *key++) << 8;
    }
    return number;
}
```

E.g. “AZF”
A = 65
Z = 90 → $90 \times 256 = 23040$
F = 70
→ `stringToInt2(“AZF”) = 23175`

溢出处理 - 示例方案

哈希函数

$h(x)$: x 第一个元素 - 'K'

$b=14, s=2$

$x = \text{LOO} \quad h(x) = 7$

$x = \text{KOO} \quad h(x) = 0$

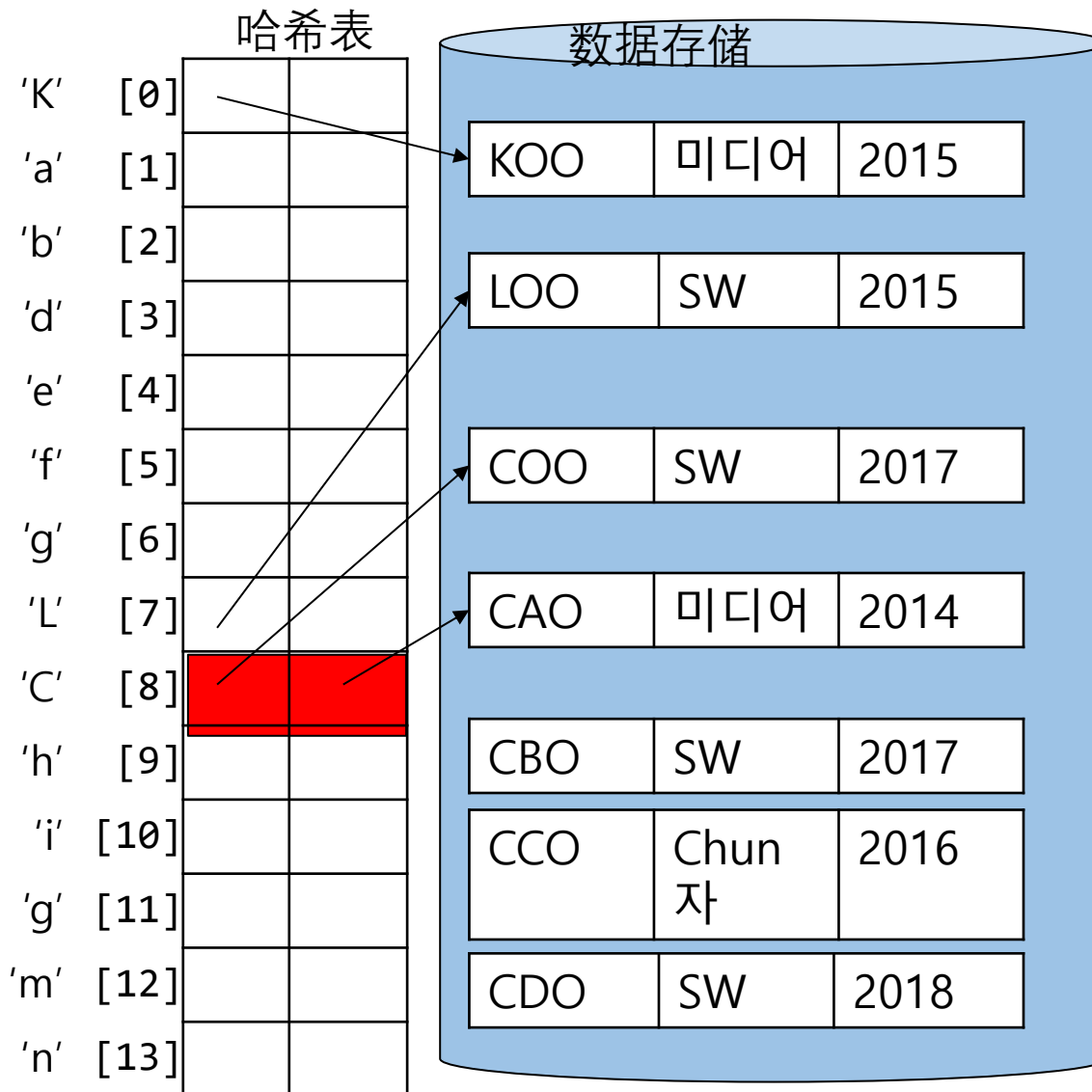
$x = \text{COO} \quad h(x) = 8$

$x = \text{CAO} \quad h(x) = 8$

$x = \text{CBO} \quad h(x) = 8$

$x = \text{CCO} \quad h(x) = 8$

$x = \text{CDO} \quad h(x) = 8$



溢流处理

❖ 开放式寻址

- 线性探查法
- 随即探查法
- 平方探查法
- 再哈希法

❖ 链地址法

- 为每个桶维护一个同义词链表

线性开放式寻址

❖ Idea

发生溢出时，找到最近的未装满的桶

❖ 伪代码

```
hIndex = hashFunction(insertKey);
found = false;
while(HT[hIndex] != emptyKey && !found){
    if(HT[hIndex].key == insertkey)
        found = true;
    else
        hIndex = (hIndex + 1) % HTSize;
}
if(found)
    printf("Duplicate items are not allowed.");
else
    HT[hIndex] = newItem;
```

线性探测示例(1)

id	加性变换	x	hash
for	102+111+114	327	2
do	100+111	211	3
while	119+104+105+108+101	537	4
if	105+102	207	12
else	101+108+115+101	425	9
function	102+117+110+99+116+105+111+110	870	12

哈希表(b=13, s=1)

[0]	function	[7]
[1]		[8]
[2]	for	[9] else
[3]	do	[10]
[4]	while	[11]
[5]		[12] if
[6]		


x mod 13

线性探测法示例(2)

哈希函数

$h(x)$: x 的第一个元素-'a'

$b=26, s=1$

Id: "acos", "atan", "char", "define",
"exp", "ceil", "cos", "float", "atol",
"floor", "ctime"

$h(\text{acos})=0$ $h(\text{atan})=0$ $h(\text{char})=2$

$h(\text{define})=3$ $h(\text{exp})=4$ $h(\text{ceil})=2$

$h(\text{cos})=2$ $h(\text{float})=5$ $h(\text{atol})=0$

$h(\text{floor})=5$ $h(\text{ctime})=2$

查找 atol? $h(\text{atol})=0$

查找 ascii? $h(\text{ascii})=0$

bucket	x	buckets searched
0	acos	1
1	atan	2
2	char	1
3	define	1
4	exp	1
5	ceil	4
6	cos	5
7	float	3
8	atol	9
9	floor	5
10	ctime	9
11		
...		
25		

线性探测：问题

❖ 随着时间的推移，探针序列会越来越长

❖ 主集群

- 键值倾向于聚集在哈希表的某一区域
- 若键值哈希到现有集群内，会被添加到集群末尾（进一步扩大集群规模）
- 其他键值若映射到该拥挤区域，也会被“卷入”集群，导致冲突率上升

Today's Topic

溢出处理(Cont.)

- 随机探测
- 平方探查法
- 链地址法

动态哈希

随机探测

- 使用随机数生成器查找下一个可用时段
- 探查序列中的第 i 个槽位为: $(h(X) + r_i) \% b$, 其中 r_i 是数值1到 $b-1$ 经过随机排列后的第 i 个值
- 所有插入和查找操作均使用相同的随机数序列

随机探查示例

哈希函数

$h(x)$: x 的第一个元素-'a'

随机数字序列:

2 4 6 1 7 9 13

$b=26, s=1$

Id: "acos", "atan",
"char", "define", "exp",
"ceil", "cos", "float",
"atol", "floor", "ctime"

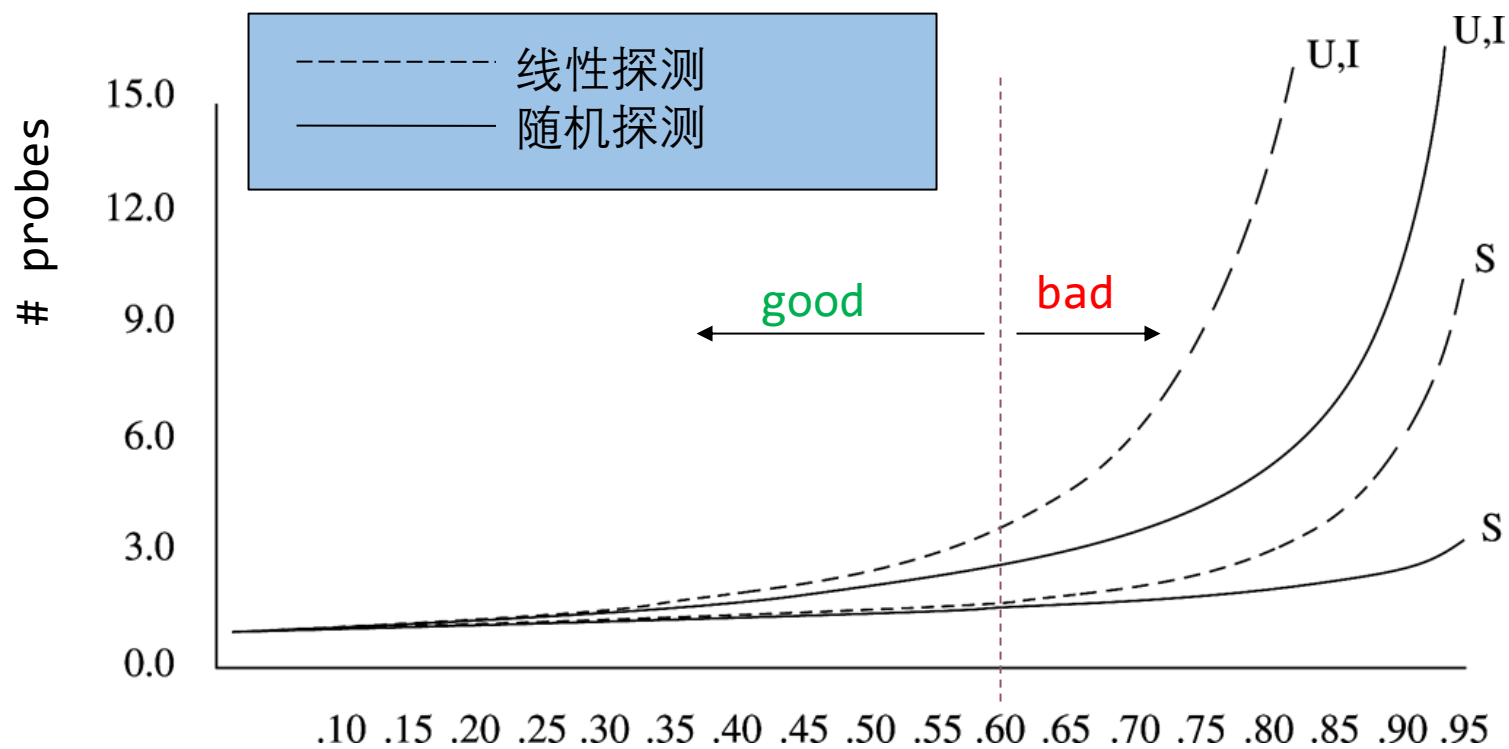
$h(\text{acos})=0$ $h(\text{atan})=0$ $h(\text{char})=2$
 $h(\text{define})=3$ $h(\text{exp})=4$ $h(\text{ceil})=2$
 $h(\text{cos})=2$ $h(\text{float})=5$ $h(\text{atol})=0$
 $h(\text{floor})=5$ $h(\text{ctime})=2$

搜寻 atol?

搜寻 ascii?

bucket	x	buckets searched
0	acos	1
1	atol	5
2	atan	2
3	define	1
4	char	2
5	float	1
6	exp	2
7	floor	2
8	ceil	4
9	cos	6
10		
11	ctime	7
...		
25		

线性 vs. 随机探测



U - 不成功查找
S - 成功查找
I - 插入

负载因子 λ

平方探测法

- 减少主群集
- 查找 $h(x), (h(x) + i^2) \% b, (h(x) - i^2) \% b$, 对于 $1 \leq i \leq (b - 1)/2$
- 如果 b 是 $4j+3$ 形式的质数, 其中 j 是整数, 则对每个桶进行检查

b	j	b	j
3	0	43	10
7	1	59	14
11	2	127	31
19	4	251	62
23	5	503	125
31	7	1019	254

平方探测法示例

哈希函数

$b=26$, $s=1$

$h(x)$: x 的的第一个元素-'a'

Id: "acos", "atan", "char", "define", "exp",
"ceil", "cos", "float", "atol", "floor",
"ctime"

$h(\text{acos})=0$ $h(\text{atan})=0$ $h(\text{char})=2$
 $h(\text{define})=3$ $h(\text{exp})=4$ $h(\text{ceil})=2$
 $h(\text{cos})=2$ $h(\text{float})=5$ $h(\text{atol})=0$
 $h(\text{floor})=5$ $h(\text{ctime})=2$

$i=1, h(\text{atan}) + 1 = 1$

$i=1, h(\text{ceil}) + 1 = 3$
 $i=2, h(\text{ceil}) + 4 = 6$

$i=1, h(\text{cos}) + 1 = 3$
 $i=2, h(\text{cos}) + 4 = 6$
 $i=3, h(\text{cos}) + 9 = 11$

bucket	X	buckets searched
0	acos	1
1	atan	2
2	char	1
3	define	1
4	exp	1
5	float	1
6	ceil	3
7		
8		
9	atol	4
10		
11	cos	4
...		
14	floor	4
...		
18	ctime	5

示例场景 - 二次探查

哈希函数

$h(x)$: x 的第一个元素 - 'k'

$x = \text{CBO}$ $h(x) = 8$

溢出

Addr $8 + 1^2$ OK

$x = \text{CCO}$ $h(x) = 8$

溢出

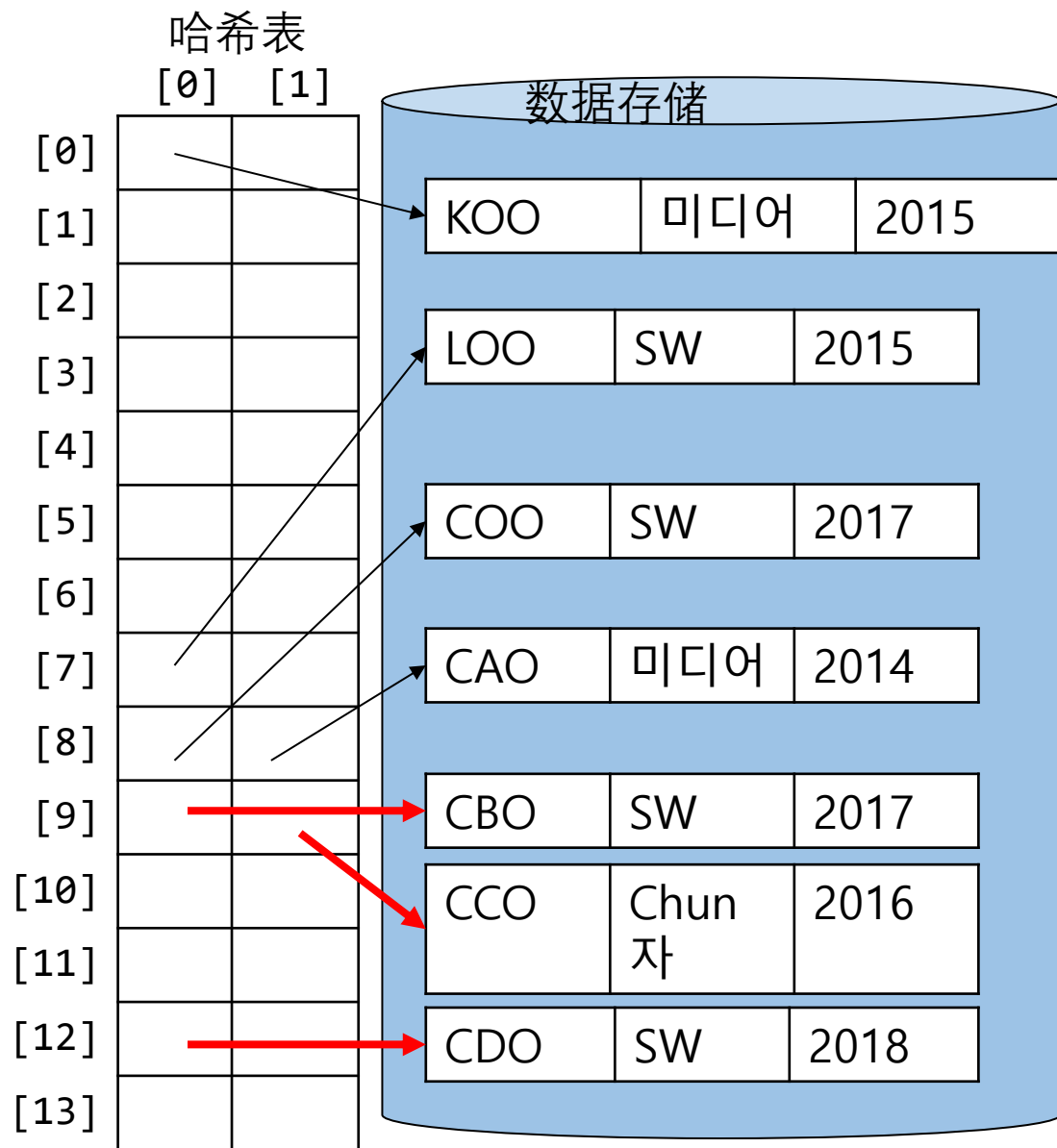
Addr $8 + 1^2$ OK

$x = \text{CDO}$ $h(x) = 8$

溢出

Addr $8 + 1^2$ 溢出

Addr $8 + 2^2$ OK



示例场景 - 二次探查

哈希函数 $h(x)$ ： x 的第一个元素-'k'

Q: 'CDO'属于哪个部门?

$x=\text{CDO}$ $h(x)=8$

Addr 8 **Fail**

Addr $8+1$ **Fail**

Addr $8+2^2$ **Success**

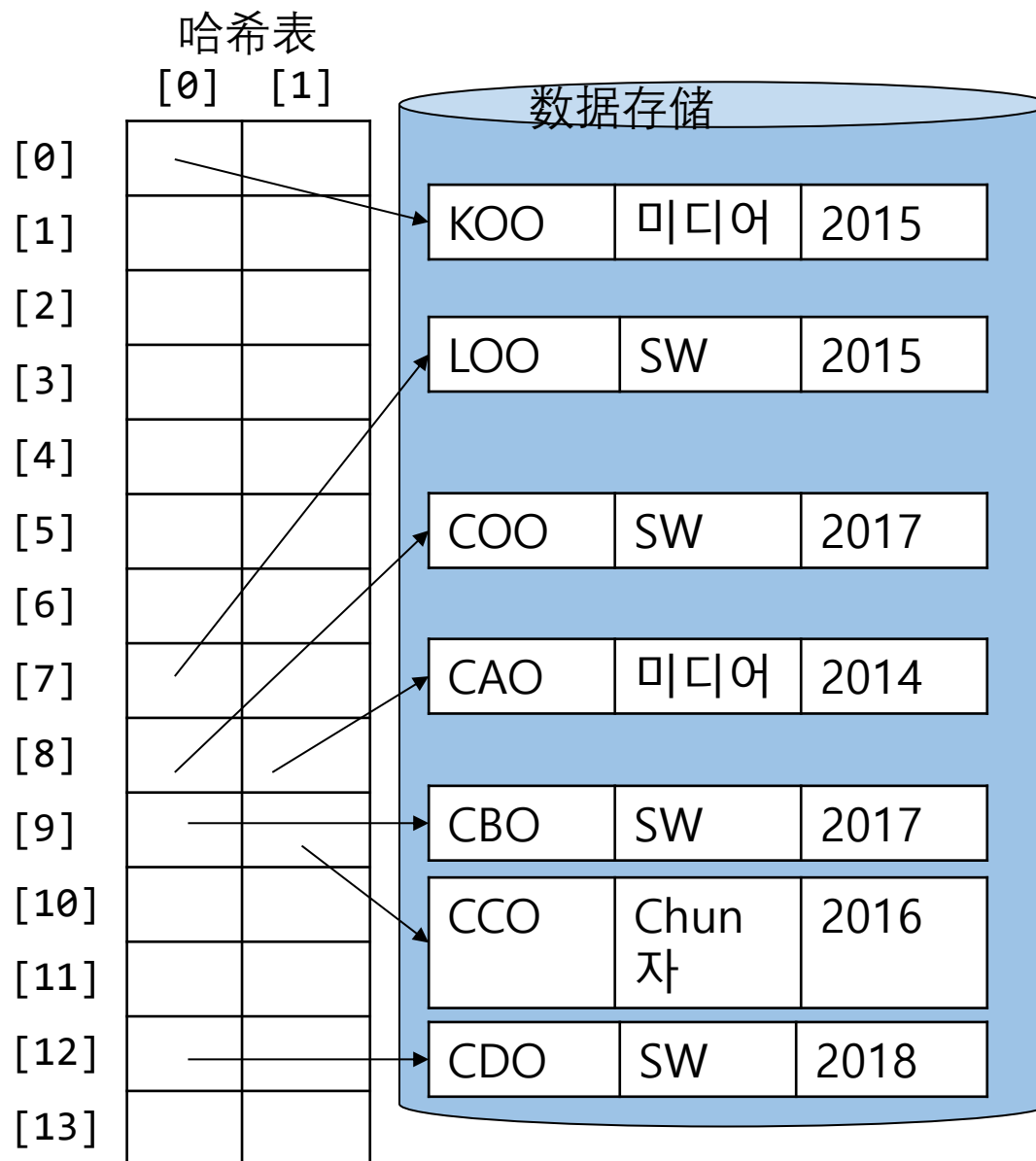
Q: 'CXO'属于哪个部门?

$x=\text{CXO}$ $h(x)=8$

Addr 8 **Fail**

Addr $8+1$ **Fail**

Addr $8+2^2$ **Fail** ➡ **Fail**



开放式寻址中的删除

❖ 两点考虑

- 删除记录不得妨碍以后的搜索
- 我们不想让哈希表中的位置因为删除而无法使用

➤ 在被删除记录的位置放置一个特殊标记，称为墓碑标记[tombstone]

若在搜索过程中遇到墓碑标记，则继续执行探查过程

在插入过程中遇到墓碑标记时，该槽位可用于存储新记录

删除示例 - 线性探测

Hash function

$h(x)$: x 的第一个元素-'k'

删除 'CBO'

查找 'CEO'??

$x=CEO$ $h(x)=8$

Addr 8 **Fail**

Addr 8+1 **Fail**

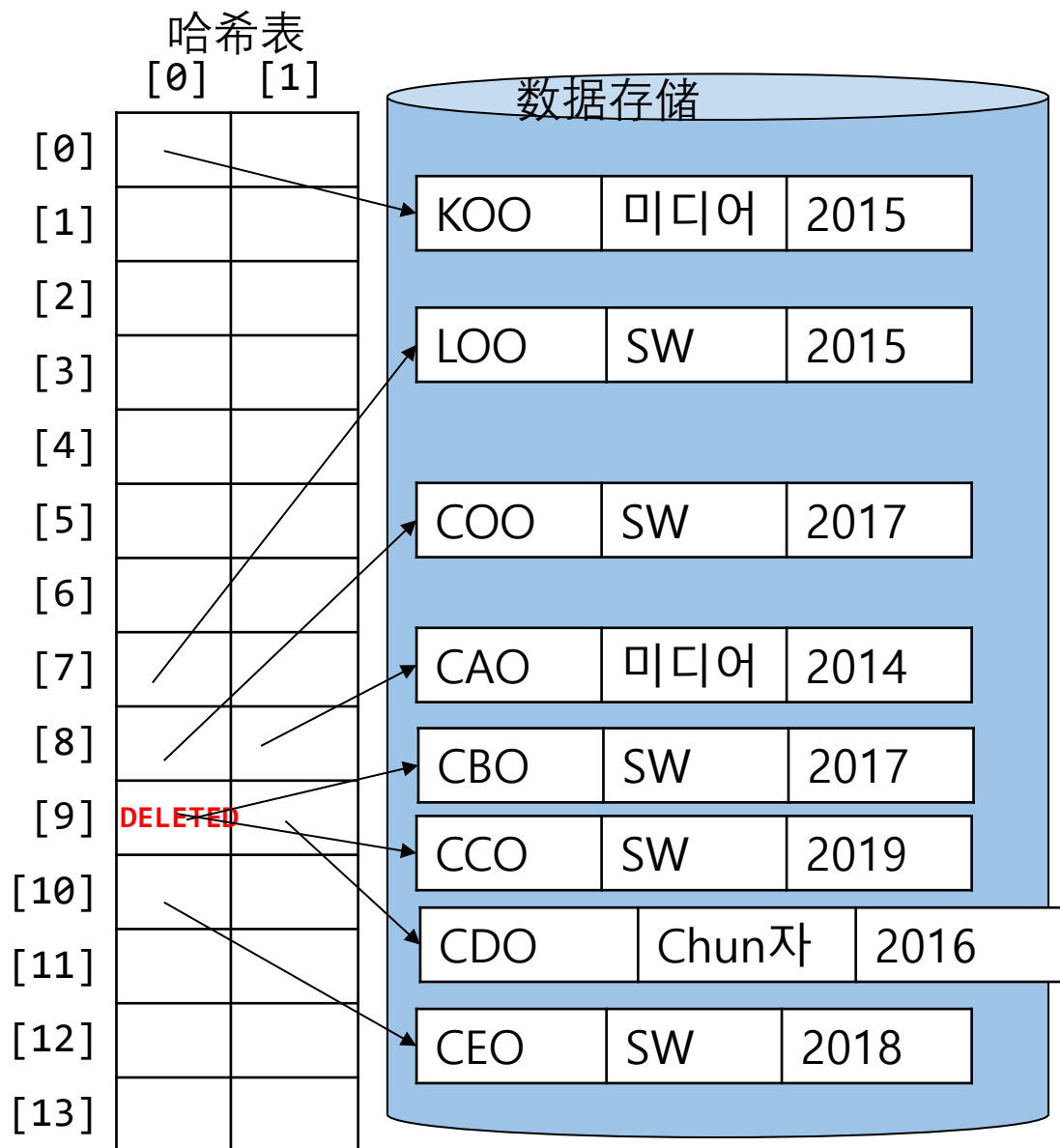
Addr 8+2 **Success**

Insert 'CCO'

$x=CCO$ $h(x)=8$

溢出

Addr 8+1 **OK**



比较

Bucket	x	Buckets searched
0	acos	1
1	atan	2
2	char	1
3	define	1
4	exp	1
5	ceil	4
6	cos	5
7	float	3
8	atol	9
9	floor	5
10	ctime	9
11		
12		
13		
14		
...		
25		

Bucket	x	Buckets searched
0	acos	1
1	atol	5
2	atan	2
3	define	1
4	char	2
5	float	1
6	exp	2
7	floor	2
8	ceil	4
9	cos	6
10		
11	ctime	7
12		
13		
14		
...		
25		

Bucket	x	Buckets searched
0	acos	1
1	atan	2
2	char	1
3	define	1
4	exp	2
5	float	1
6	ceil	3
7		
8		
9	atol	4
10		
11	cos	4
12		
13		
14	floor	4
15		
16		
17		
18	ctime	5
...		
25		

链地址法(1)

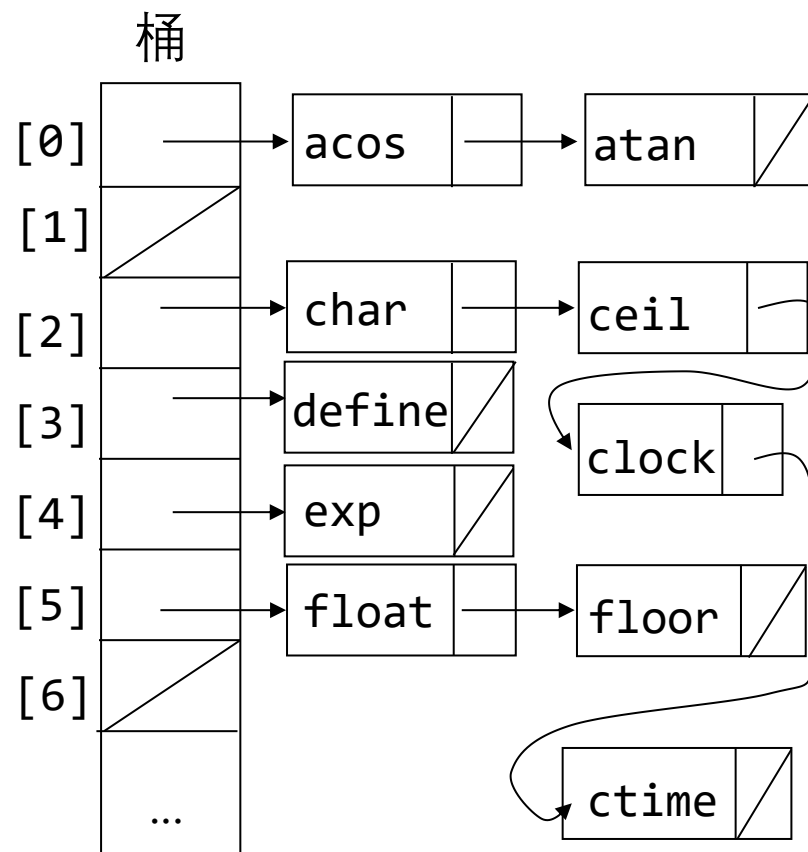
为每个桶维护一个同义词链接列表 $\rightarrow s$ 具有灵活性

哈希函数

$h(x)$: x 的第一个元素 - 'a'

Id: "acos", "define", "float", "exp",
"char", "atan", "ceil", "floor",
"clock", "ctime"

$h(\text{acos}) = h(\text{atan}) = 0$, $h(\text{define}) = 3$, $h(\text{exp}) = 4$,
 $h(\text{char}) = h(\text{ceil}) = h(\text{clock}) = h(\text{ctime}) = 2$,
 $h(\text{float}) = h(\text{floor}) = 5$



链地址法的潜在缺点

- ❖ 链接列表[Linked lists]可能会变得很长
 - 尤其是当 N 接近 M 时 (N = 哈希表中元素数量, M = 哈希表大小)
 - 较长的链表会对性能产生负面影响
- ❖ 由于指针的存在需要更多内存
- ❖ 绝对最坏情况 (即使 $N \ll M$)
 - 所有 N 个元素位于同一个链表中!
 - 这通常是由于糟糕的哈希函数导致的

动态哈希

随着数据库规模增长，我们有三种扩展策略：

1. 基于当前文件大小选择哈希函数
 - 随着文件增长，性能逐渐下降
2. 基于预期文件大小选择哈希函数
 - 初期会浪费空间
3. 定期重组哈希结构以适应增长
 - 代价高昂，且需暂停数据库服务

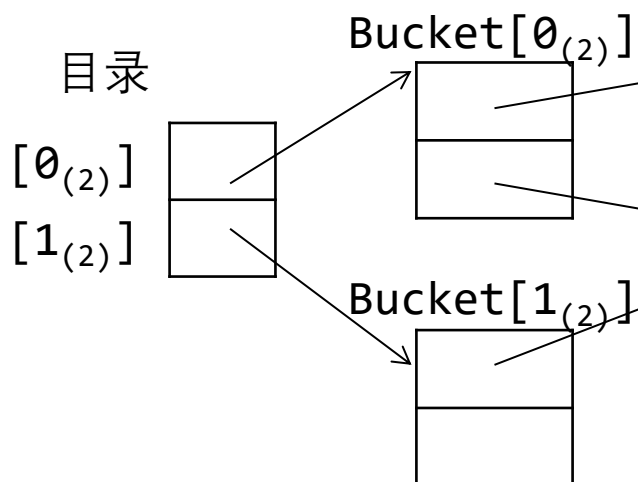
可扩展的哈希

- ❖ Bucket（主页面）变满→将数据桶数量增加一倍
- ❖ Idea: 使用指向数据桶的指针目录,
 - 通过将目录翻倍, 使桶的数量翻倍
 - 分割溢出的数据桶
- ❖ 目录比文件小得多, 因此翻倍的成本要低得多。只分割一页数据条目。
- ❖ 诀窍在于如何调整哈希函数.

动态哈希示例 - 数据插入(1)

哈希函数 $h(x) = f(x) \bmod 10_{(2)}$

$b=2$ $s=2$



$x=LeeOO$ $f(x)=011101$ $h(x)=1$

$x=KimOO$ $f(x)=010010$ $h(x)=0$

$x=ChoOO$ $f(x)=101000$ $h(x)=0$

$x=ChungOO$ $f(x)=011010$ $h(x)=0$

溢出! → 目录加倍!!

动态哈希示例 - 数据插入(2)

哈希函数 $h(x) = f(x) \bmod 10_{(2)}$

哈希函数 $h_2(x) = f(x) \bmod 100_{(2)}$

目录

$[00_{(2)}]$

$[01_{(2)}]$

$[10_{(2)}]$

$[11_{(2)}]$

Bucket $[00_{(2)}]$

Bucket $[10_{(2)}]$

Bucket $[01_{(2)}]$

Bucket $[11_{(2)}]$

插入 LeeOO!
插入 KimOO!
插入 ChoOO!
插入 ChungOO!

$x=LeeOO$ $f(x)=011101$ $h_2(x)=01$

$x=KimOO$ $f(x)=010010$ $h_2(x)=10$

$x=ChoOO$ $f(x)=101000$ $h_2(x)=00$

$x=ChungOO$ $f(x)=011010$ $h_2(x)=10$

数据存储1

KimOO | 미디어 | 2015

LeeOO | SW | 2015

ChoOO | SW | 2017

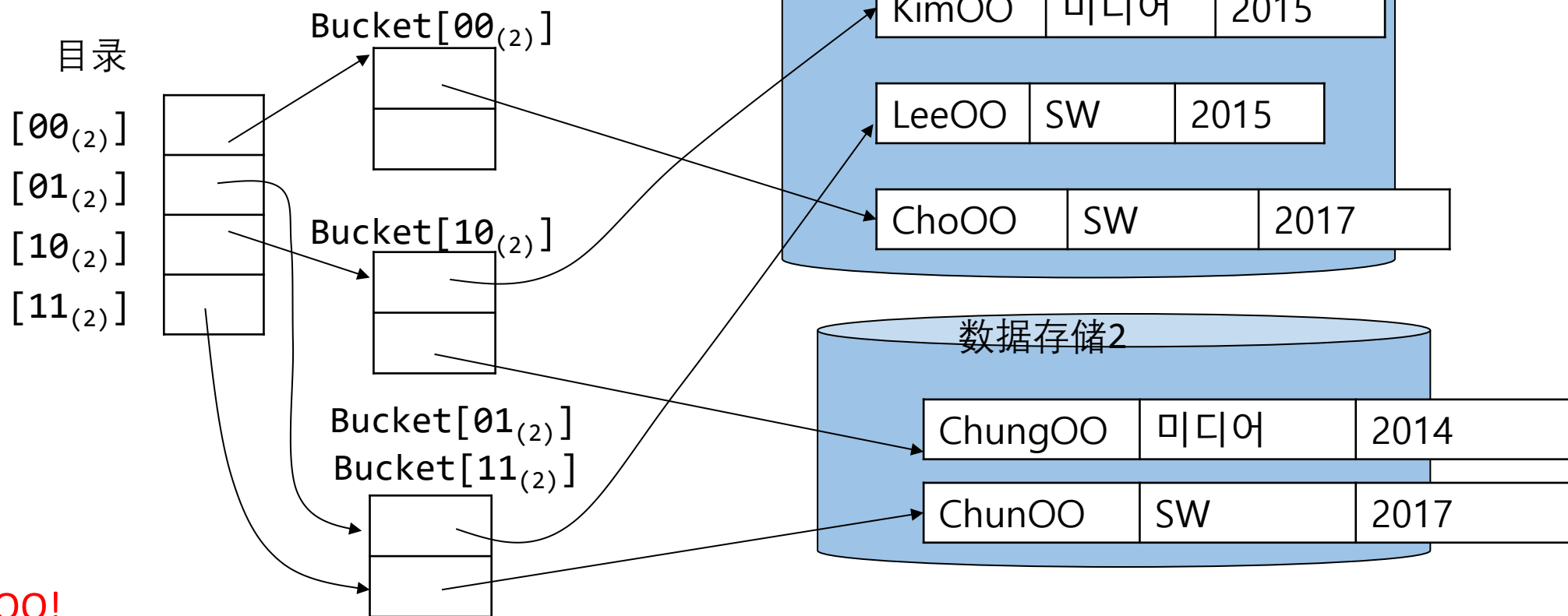
数据存储2

ChungOO | 미디어 | 2014

动态哈希示例 - 数据插入(3)

哈希函数 $h(x) = f(x) \bmod 10_{(2)}$

哈希函数 $h_2(x) = f(x) \bmod 100_{(2)}$



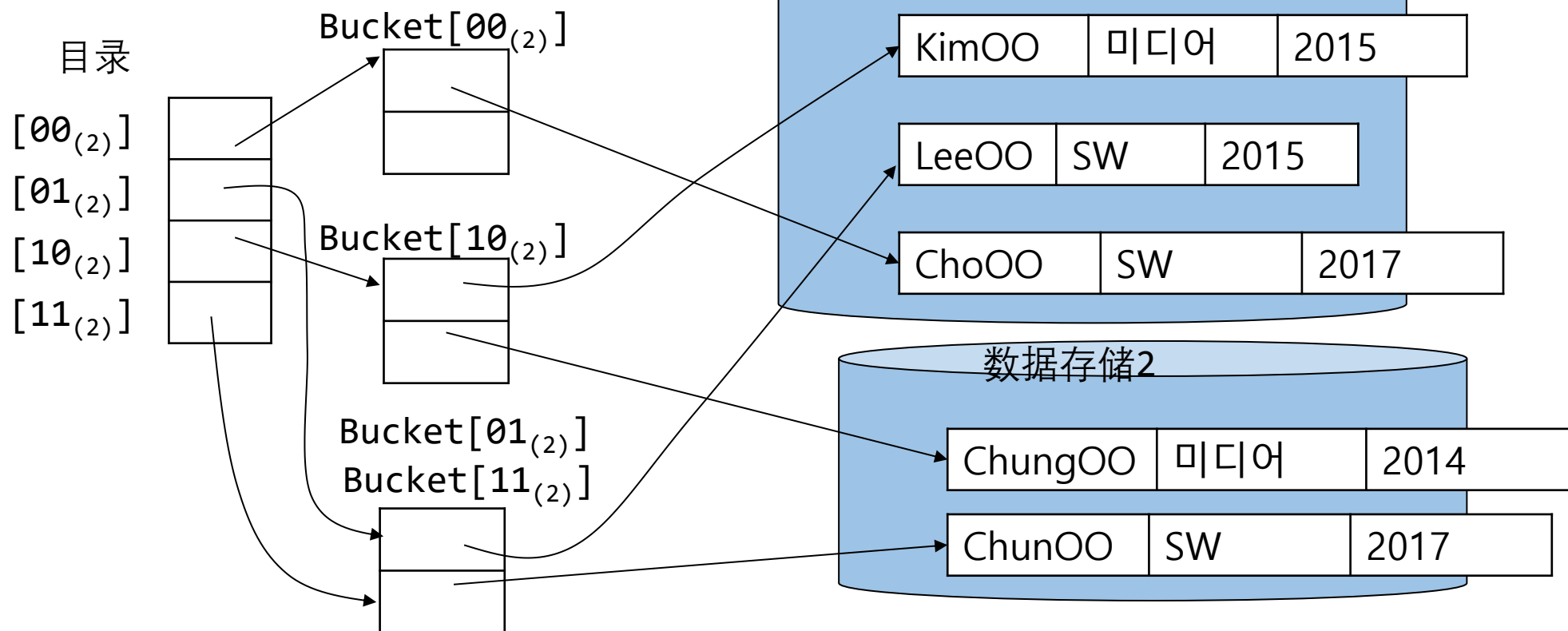
插入 ChunOO!

$x=\text{ChunOO}$ $f(x)= 001011$ $h_2(x)= 11$

动态哈希示例 - 数据搜索

哈希函数 $h(x) = f(x) \bmod 10_{(2)}$

哈希函数 $h_2(x) = f(x) \bmod 100_{(2)}$



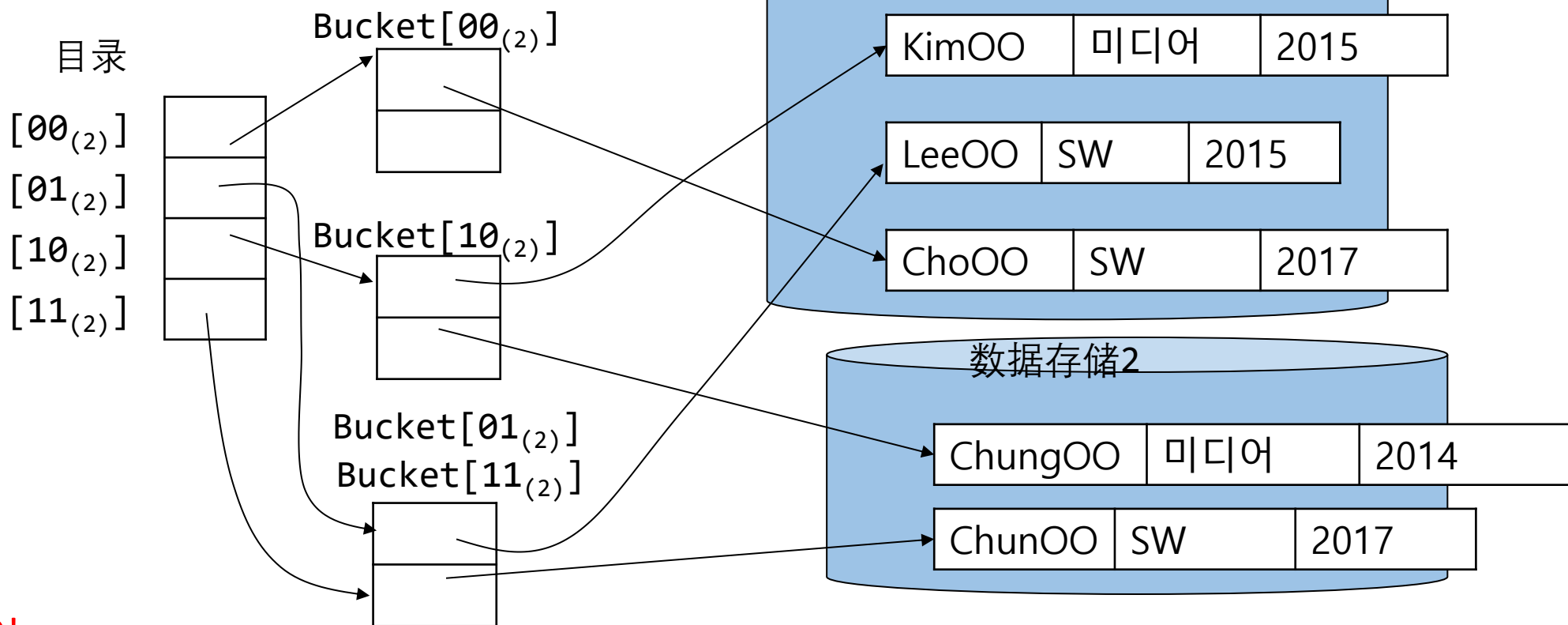
'LeeOO'의学院?? $x=LeeOO$ $f(x)=011101$ $h_2(x)=01$

'ChunOO'의学院?? $x=ChunOO$ $f(x)=001011$ $h_2(x)=11$

动态哈希示例 - 数据插入(4)

哈希函数 $h(x) = f(x) \bmod 10_{(2)}$

哈希函数 $h_2(x) = f(x) \bmod 100_{(2)}$



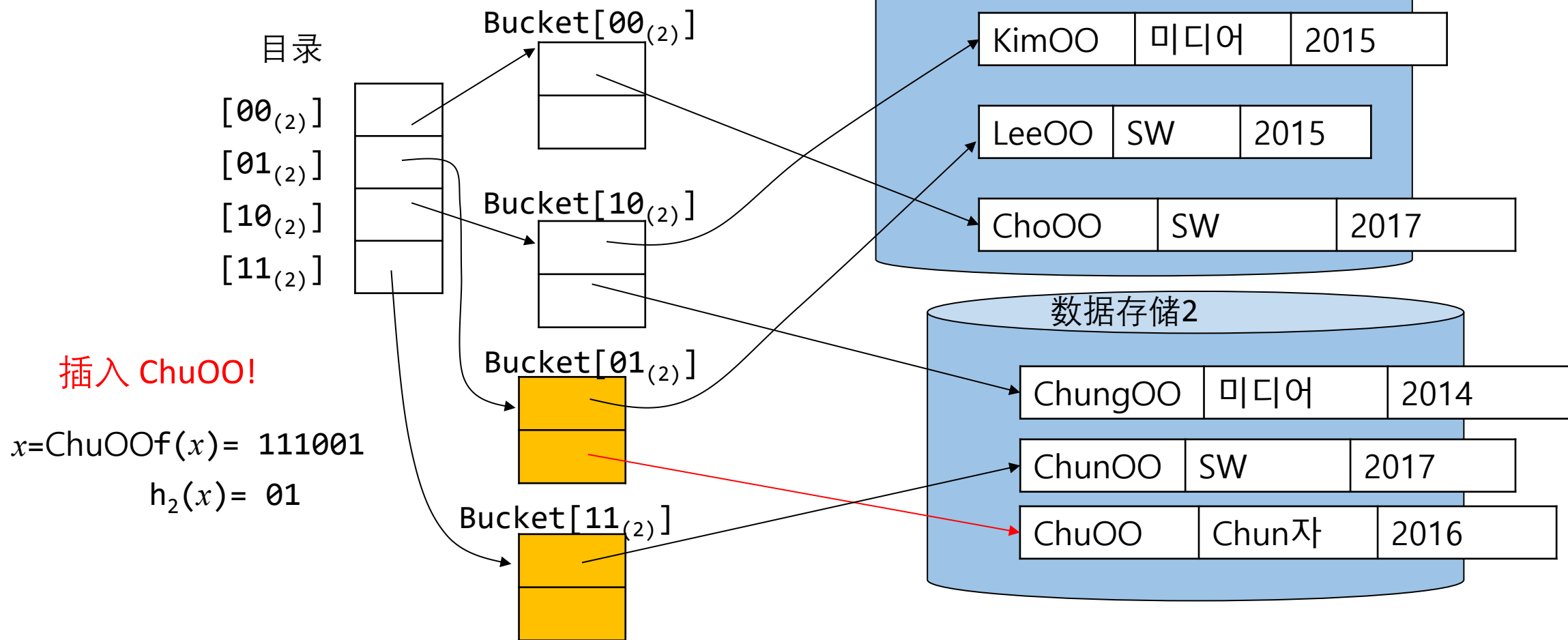
插入 ChuOO!

$x=ChuOO$ $f(x)= 111001$ $h_2(x)= 01$ 溢出!!

动态哈希示例 - 数据插入(5)

哈希函数 $h(x) = f(x) \bmod 10_{(2)}$

哈希函数 $h_2(x) = f(x) \bmod 100_{(2)}$



无目录动态哈希

❖ 设计动机

- 可扩展哈希的局限性
- 能否实现更平滑的扩容 (Smoother Growth) , 避免目录的剧烈变动?
- 线性哈希从左到右拆分数数据桶, 而不管哪个数据桶溢出
- 不使用目录, 而是使用溢出桶 (链式方法)

线性哈希示例(1)

初始化: $h(x) = x \bmod N$ ($N=4$ here)

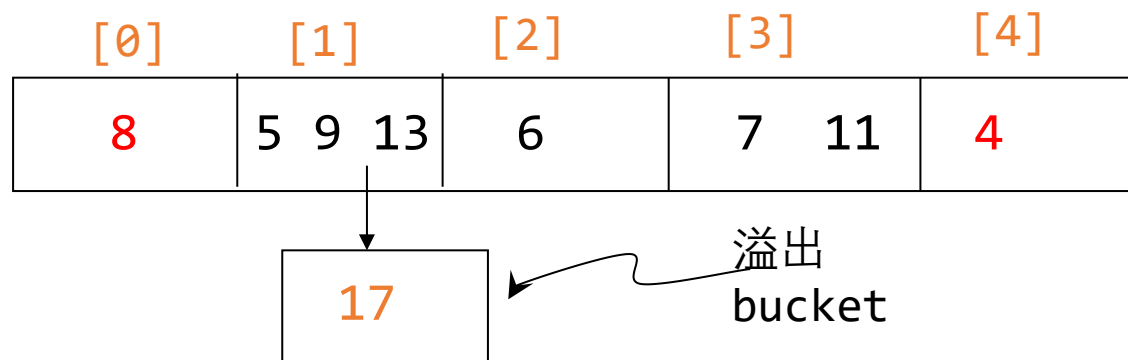
假设: 3 records/bucket (负载因子)

[0]	[1]	[2]	[3]
4 8	5 9 13	6	7 11

插入 17 = $17 \bmod 4 = 1$

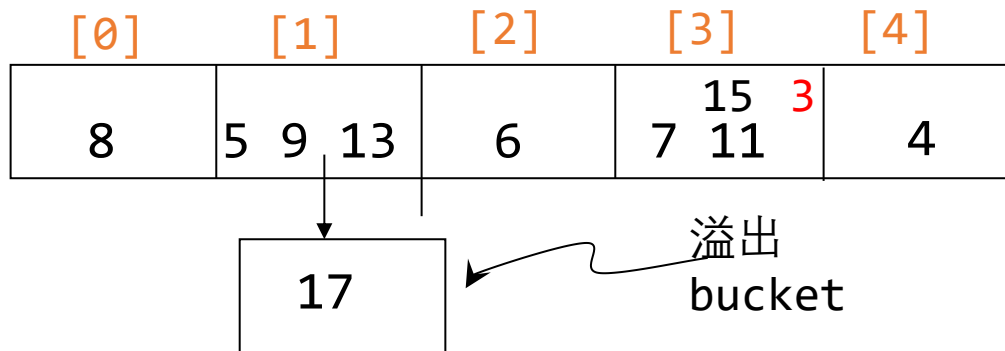
- bucket 1 溢出
- 分割 bucket 0
- 应用新哈希函数

$$h_0(x) = x \bmod N, \quad h_1(x) = x \bmod 2 \times N$$



线性哈希示例 (2)

$$h_0(x) = x \bmod N, \quad h_1(x) = x \bmod 2 \times N$$

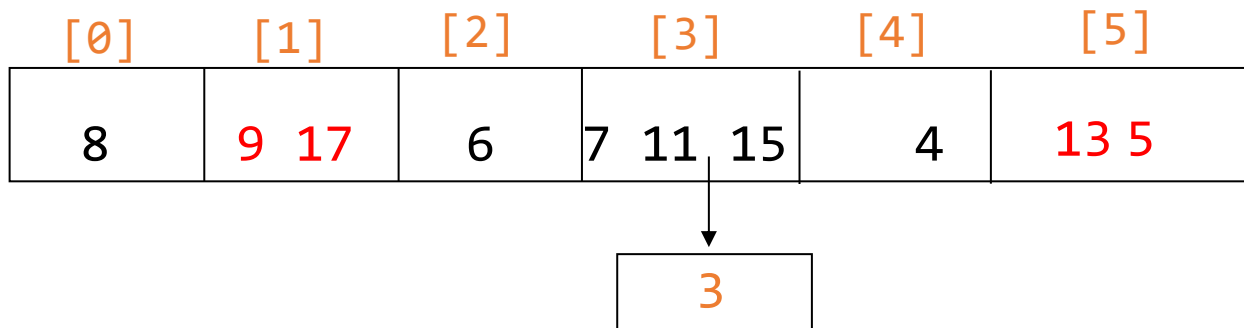


插入 15 和 3 $\rightarrow h_0(15) = 3, h_0(3) = 3$

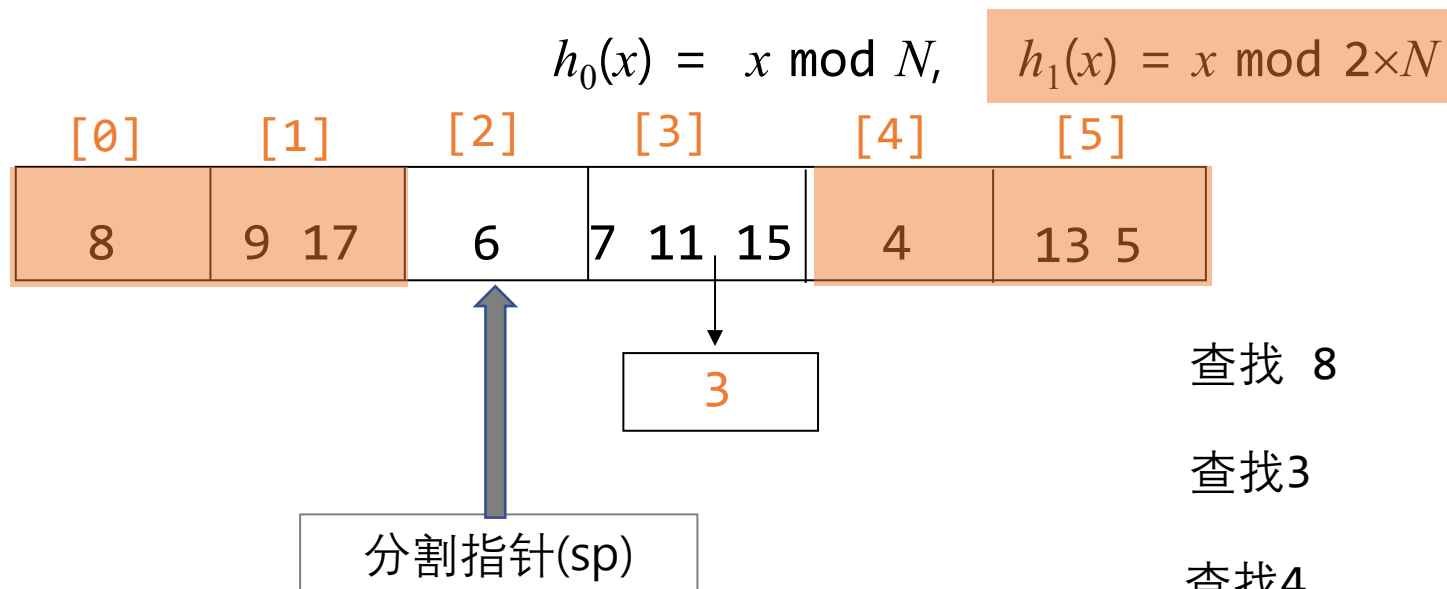
$\rightarrow$ 溢出 bucket 3

$\rightarrow$ 分割 bucket 1

$$h_0(x) = x \bmod N, \quad h_1(x) = x \bmod 2 \times N$$



线性哈希示例 (3)



查找 8	$h_0(8)=0$	$h_1(8)=0$	Success!
查找 3	$h_0(3)=3$		Success!
查找 4	$h_0(4)=0$	$h_1(4)=4$	Success!
查找 20	$h_0(20)=0$	$h_1(20)=4$	Fail!

❖ 查找代码

如果 $(h_0(\text{key}) < \text{sp})$ ，则搜索是从 bucket $h_1(\text{key})$ 开始的链；
否则，搜索 bucket $h_0(\text{key})$ 开始的链；

Next Topic

查找结构

- AVL T树

- 红黑树